

Bloc 2 TP 8

Contexte	2
Exercice 1	2
Exercice 2	4
Exercice 3	6
Exercice 4	7
Exercice 5	9
Exercice 6	11
Exercice 7	12
Exercice 8	12
Exercice 9	14
Exercice 10	15
Exercice 11	16
Exercice 12	19
Exercice 13	20
Exercice 14	22
Exercice 15	23
Exercice 16	24
Exercice 17	25
Exercice 18	26
Exercice 19	27
Exercice 20	28
Bonus	28
Conclusion	28

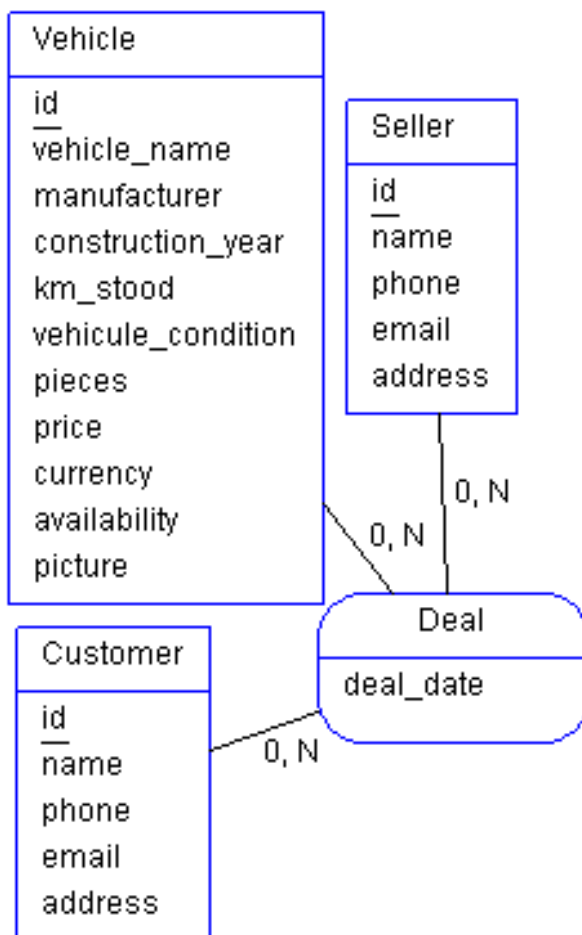
Contexte

Ce TP a pour objectif de développer une application de bureau en Java (JavaFX) pour la gestion d'un concessionnaire automobile. L'enjeu est de relier une interface graphique à une base de données MySQL pour effectuer des opérations CRUD (Créer, Lire, Modifier, Supprimer) sur les véhicules, les clients et les vendeurs, tout en intégrant la gestion des images.

(Les fichiers IntelliJ, .sql et .jar sont disponible dans le portfolio)

Exercice 1

MCD :



Création de la BDD :

```
CREATE DATABASE IF NOT EXISTS cardealer;
```

```
USE cardealer;
```

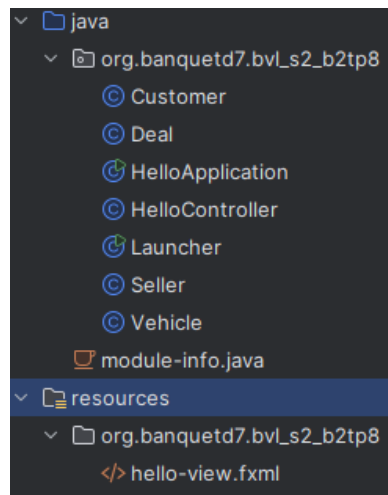
```
CREATE TABLE Vehicle ( vehicle_id INT AUTO_INCREMENT  
PRIMARY KEY, vehicle_name VARCHAR(255) NOT NULL,  
manufacturer VARCHAR(255) NOT NULL, construction_year  
VARCHAR(4) NOT NULL, km_stood DOUBLE NOT NULL,  
vehicle_condition VARCHAR(50) NOT NULL, pieces INT NOT  
NULL, price DOUBLE NOT NULL, currency VARCHAR(10) NOT  
NULL, availability VARCHAR(50) NOT NULL, picture  
MEDIUMBLOB );
```

```
CREATE TABLE Customer ( customer_id INT AUTO_INCREMENT  
PRIMARY KEY, name VARCHAR(255) NOT NULL, phone  
VARCHAR(20) NOT NULL, email VARCHAR(255) NOT NULL,  
address VARCHAR(255) NOT NULL );
```

```
CREATE TABLE Seller ( seller_id INT AUTO_INCREMENT  
PRIMARY KEY, name VARCHAR(255) NOT NULL, phone  
VARCHAR(20) NOT NULL, email VARCHAR(255) NOT NULL,  
address VARCHAR(255) NOT NULL );
```

```
CREATE TABLE Deal ( deal_id INT AUTO_INCREMENT PRIMARY  
KEY, vehicle_id INT NOT NULL, customer_id INT NOT NULL,  
seller_id INT NOT NULL, deal_date TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP, FOREIGN KEY (vehicle_id) REFERENCES  
Vehicle(vehicle_id), FOREIGN KEY (customer_id)  
REFERENCES Customer(customer_id), FOREIGN KEY  
(seller_id) REFERENCES Seller(seller_id) );
```

Exercice 2



Créations des classes avec constructeur, getter et setter :

```
package org.banquetd7.bvl_s2_b2tp8;

public class Customer {
    private int customerId; 3 usages
    private String name; 3 usages
    private String phone; 3 usages
    private String email; 3 usages
    private String address; 3 usages

    public Customer() {
    }

    public Customer(int customerId, String name, String phone, String email, String address) {
        this.customerId = customerId;
        this.name = name;
        this.phone = phone;
        this.email = email;
        this.address = address;
    }

    public int getCustomerId() { return customerId; } no usages
    public void setCustomerId(int customerId) { this.customerId = customerId; } no usages

    public String getName() { return name; } no usages
    public void setName(String name) { this.name = name; } no usages

    public String getPhone() { return phone; } no usages
    public void setPhone(String phone) { this.phone = phone; } no usages

    public String getEmail() { return email; } no usages
    public void setEmail(String email) { this.email = email; } no usages

    public String getAddress() { return address; } no usages
    public void setAddress(String address) { this.address = address; } no usages
}
```

```

package org.banquetd7.bvl_s2_b2tp8;

import java.sql.Timestamp;

public class Deal {
    private int dealId; 3 usages
    private int vehicleId; 3 usages
    private int customerId; 3 usages
    private int sellerId; 3 usages
    private Timestamp dealDate; 3 usages

    public Deal() {
    }

    public Deal(int dealId, int vehicleId, int customerId, int sellerId, Timestamp dealDate) {
        this.dealId = dealId;
        this.vehicleId = vehicleId;
        this.customerId = customerId;
        this.sellerId = sellerId;
        this.dealDate = dealDate;
    }

    public int getDealId() { return dealId; } no usages
    public void setDealId(int dealId) { this.dealId = dealId; } no usages

    public int getVehicleId() { return vehicleId; } no usages
    public void setVehicleId(int vehicleId) { this.vehicleId = vehicleId; } no usages

    public int getCustomerId() { return customerId; } no usages
    public void setCustomerId(int customerId) { this.customerId = customerId; } no usages

    public int getSellerId() { return sellerId; } no usages
    public void setSellerId(int sellerId) { this.sellerId = sellerId; } no usages

    public Timestamp getDealDate() { return dealDate; } no usages
    public void setDealDate(Timestamp dealDate) { this.dealDate = dealDate; } no usages
}

```

Exercice 3

Création de la page principale :

```
@Override
public void start(Stage stage) throws IOException {
    FXMLLoader fxmlLoader = new FXMLLoader(HelloApplication.class.getResource("hello-view.fxml"));
    Scene scene = new Scene(fxmlLoader.load(), 1200, 900);

    stage.setTitle("Welcome");
    try {
        stage.getIcons().add(new Image(HelloApplication.class.getResourceAsStream("icon.png")));
    } catch (NullPointerException e) {
        System.out.println("Icône introuvable. Poursuite de l'exécution sans icône.");
    }

    stage.setScene(scene);
    stage.show();
}
```

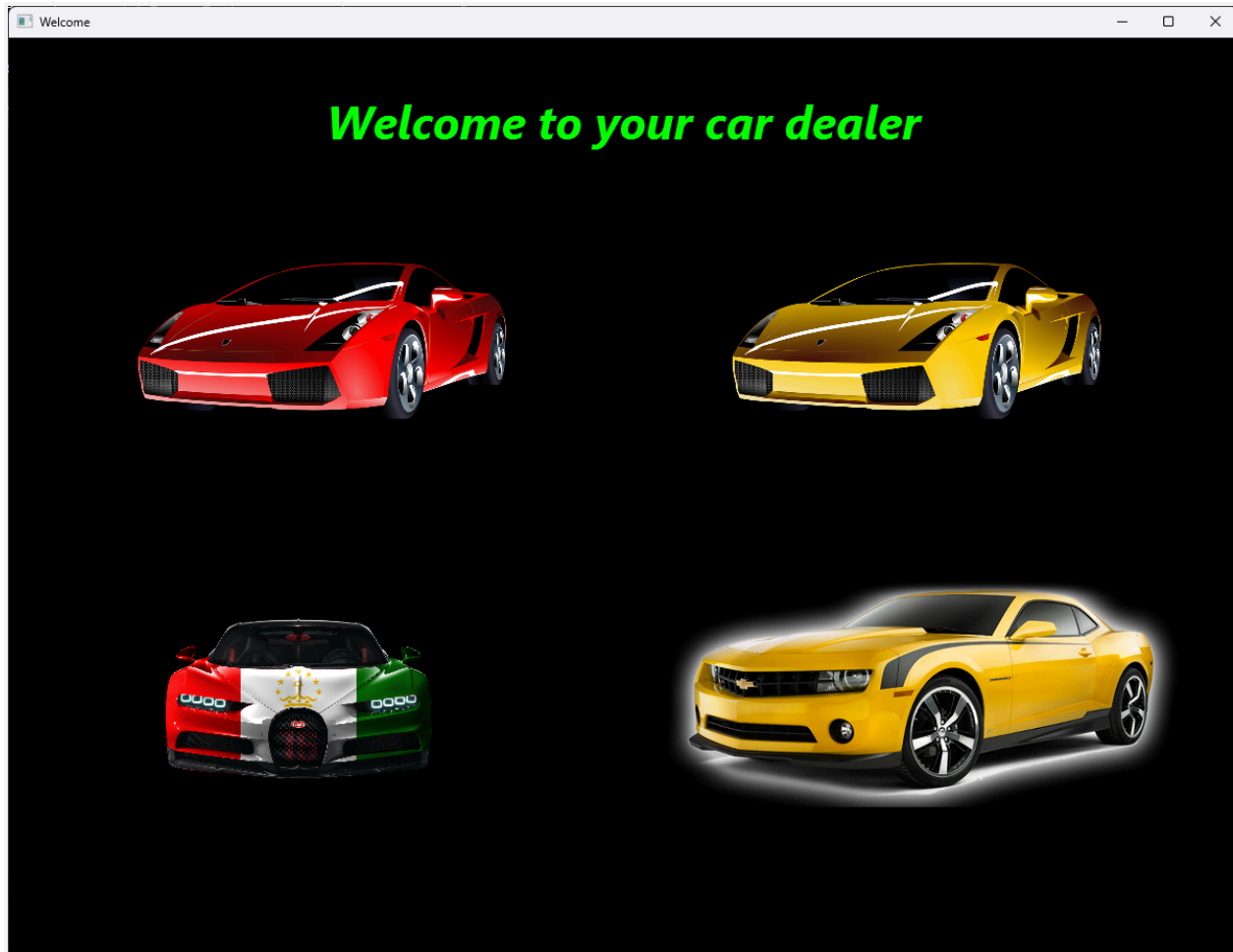
```
<AnchorPane prefHeight="900.0" prefWidth="1200.0" style="-fx-background-color: black;" xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1">
    <children>
        <Label alignment="CENTER" layoutX="0.0" layoutY="50.0" prefHeight="60.0" prefWidth="1200.0" text="Welcome to your car dealer" textAlignment="center">
            <font>
                <Font name="System Bold Italic" size="45.0" />
            </font>
        </Label>

        <ImageView fitHeight="250.0" fitWidth="500.0" layoutX="60.0" layoutY="180.0" pickOnBounds="true" preserveRatio="true">
            <image>
                <Image url="@../../../../image_cardealer/ICONS/lamborghini-Red.png" />
            </image>
        </ImageView>

        <ImageView fitHeight="250.0" fitWidth="500.0" layoutX="640.0" layoutY="180.0" pickOnBounds="true" preserveRatio="true">
            <image>
                <Image url="@../../../../image_cardealer/ICONS/lamborghini.Yellow.png" />
            </image>
        </ImageView>

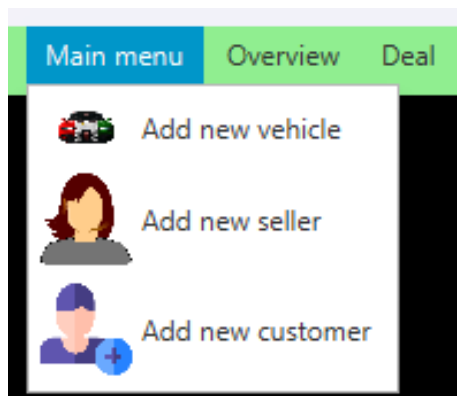
        <ImageView fitHeight="250.0" fitWidth="500.0" layoutX="60.0" layoutY="520.0" pickOnBounds="true" preserveRatio="true">
            <image>
                <Image url="@../../../../image_cardealer/ICONS/Bugatti-new.png" />
            </image>
        </ImageView>

        <ImageView fitHeight="250.0" fitWidth="500.0" layoutX="640.0" layoutY="520.0" pickOnBounds="true" preserveRatio="true">
            <image>
                <Image url="@../../../../image_cardealer/ICONS/Chevrolet.png" />
            </image>
        </ImageView>
    </children>
</AnchorPane>
```



Exercice 4

Création du **MenuBar** :



```

<MenuBar fx:id="mymenubar" prefHeight="30.0" prefWidth="1200.0" style="-fx-background-color: lightgreen;">
    <menus>
        <Menu mnemonicParsing="false" text="Main menu">
            <items>
                <MenuItem mnemonicParsing="false" text="Add new vehicle">
                    <graphic>
                        <ImageView fitHeight="40.0" fitWidth="40.0" pickOnBounds="true" preserveRatio="true">
                            <image>
                                <Image url="@../../../../image_cardealer/ICONS/Bugatti-new.png" />
                            </image>
                        </ImageView>
                    </graphic>
                </MenuItem>
                <MenuItem mnemonicParsing="false" text="Add new seller">
                    <graphic>
                        <ImageView fitHeight="40.0" fitWidth="40.0" pickOnBounds="true" preserveRatio="true">
                            <image>
                                <Image url="@../../../../image_cardealer/ICONS/woman.png" />
                            </image>
                        </ImageView>
                    </graphic>
                </MenuItem>
                <MenuItem mnemonicParsing="false" text="Add new customer">
                    <graphic>
                        <ImageView fitHeight="40.0" fitWidth="40.0" pickOnBounds="true" preserveRatio="true">
                            <image>
                                <Image url="@../../../../image_cardealer/Meine Symbole/add-user.png" />
                            </image>
                        </ImageView>
                    </graphic>
                </MenuItem>
            </items>
        </Menu>
        <Menu mnemonicParsing="false" text="Overview" />
        <Menu mnemonicParsing="false" text="Deal" />
    </menus>
</MenuBar>

```


Exercise 5

Création d'une nouvelle classe **NewVehicle** et d'une page "addnewvehicle"

The screenshot shows a software application window titled "Add new Vehicle". The window has a dark background. At the top center, there is a car image with a red, white, and green livery. Below the image, there is a form with the following fields:

- Vehicle name:
- Manufacturer:
- Construction year:
- Km stood:
- Condition:
- Number of pieces:
- Price:
- Currency:

At the bottom of the form, there are three buttons:

-
-
-

On the left side of the application window, there is a sidebar with a list of controls and a search bar. The sidebar includes a search bar with the text "empty()", a list of controls including "Controls", "Menu", "Miscellaneous", "Shapes", "Charts", and "3D", and a search bar with the text "empty()".

```
</> addnewvehicle.fxml
</> hello-view.fxml
```

```

package org.banquetd7.bvL_s2_b2tp8;

import javafx.fxml.FXML;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.scene.control.TextField;
import javafx.scene.image.ImageView;
import javafx.scene.layout.AnchorPane;

public class NewVehicle {

    @FXML
    private AnchorPane myanchorpane;

    @FXML
    private TextField tf_vehicle_name;
    @FXML
    private TextField tf_manufacturer;
    @FXML
    private TextField tf_construction_year;
    @FXML
    private TextField tf_km_stood;
    @FXML
    private TextField tf_pieces;
    @FXML
    private TextField tf_price;

    @FXML
    private ComboBox<String> cb_condition;
    @FXML
    private ComboBox<String> cb_currency;

    @FXML
    private ImageView imageview_add_new_vehicle;

    @FXML
    private Button button_upload_picture;
    @FXML
    private Button button_save;
    @FXML
    private Button button_main_menu;

```

Exercise 6

Mise à jour de HelloController :

```
public class HelloController implements Initializable {

    @FXML
    private MenuBar mymenubar;

    @FXML // usage
    public MenuItem menuitem_addnewvehicule;

    @Override
    public void initialize(URL url, ResourceBundle resourceBundle) {
        menuitem_addnewvehicule.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                FXMLLoader loader = new FXMLLoader(HelloApplication.class.getResource("name: addnewvehicule.fxml"));

                try {
                    Parent root = loader.load();
                    Stage stage = (Stage) mymenubar.getScene().getWindow();
                    stage.setScene(new Scene(root, 1200, 900));
                    stage.setMaxHeight(900);
                    stage.setMaxWidth(1200);
                    stage.setTitle("add new vehicle");
                    stage.show();
                } catch (IOException e) {
                    throw new RuntimeException(e);
                }
            }
        });
    }
}
```

Puis de *hello-view.fxml* :

```
<MenuItem fx:id="menuitem_addnewvehicule" mnemonicParsing="false" text="Add new vehicle">
```



Vehicle name	582741
Manufacturer	75841
Construction year	63825410
Km stood	698527410
Condition	Please select Vehicle condition
Number of pieces	41
Price	41
Currency	Please select currency

Upload Vehicle Picture Save Main menu

Exercice 7

Ajout de la connexion à la BDD :

```
package org.banquetd7.bvl_s2_b2tp8;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

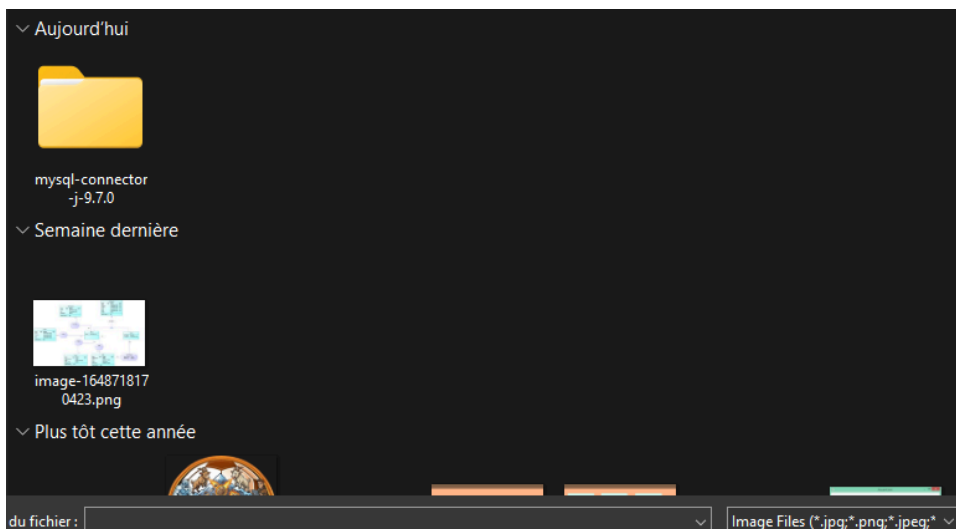
    public static Connection getConnection() throws SQLException { no usages
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
            throw new SQLException("MySQL JDBC driver not found", e);
        }
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/cardealer", "user: root", "password: ");
    }
}
```

Clique droit -> **Add as Library...**



Exercice 8

Création de l'upload de l'image :



A partir de ceci :

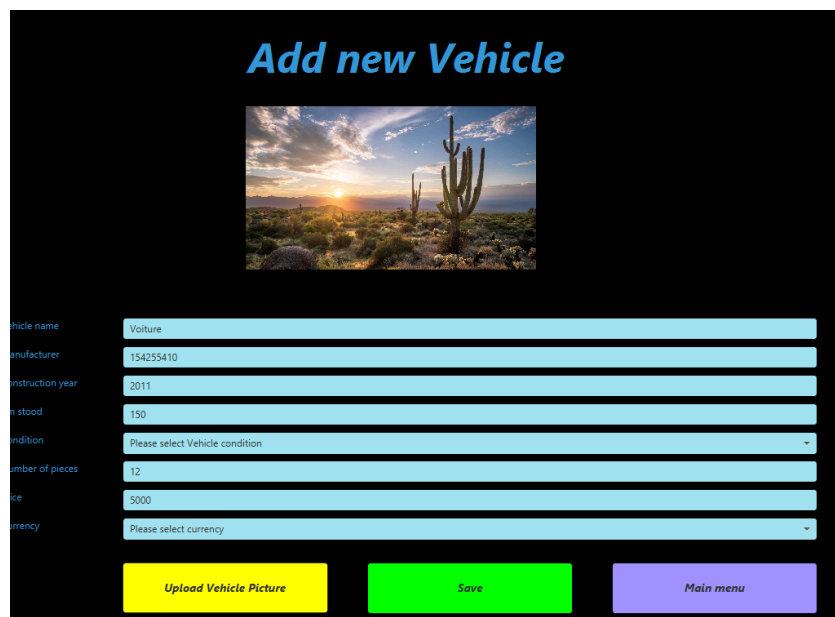
```
public class NewVehicle implements Initializable {
    @FXML
    private Button button_upload_picture;
    @FXML
    private Button button_save;
    @FXML
    private Button button_main_menu;
    String path; 3 usages

    @Override
    public void initialize(URL url, ResourceBundle resourceBundle) {
        button_upload_picture.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                FileChooser fileChooser = new FileChooser();
                fileChooser.setTitle("Upload Vehicle Picture");

                fileChooser.getExtensionFilters().addAll(
                    new FileChooser.ExtensionFilter("Image Files", ...strings: "*.jpg", "*.png", "*.jpeg", "*.svg")
                );

                Stage stage = (Stage) myanchorpane.getScene().getWindow();
                File file = fileChooser.showOpenDialog(stage);
                if (file == null) {
                    return;
                } else {
                    try {
                        path = file.getAbsolutePath();
                        System.out.println(path);
                        Image image = new Image(new FileInputStream(path));
                        imageview_add_new_vehicle.setImage(image);
                    } catch (FileNotFoundException e) {
                        throw new RuntimeException(e);
                    }
                }
            }
        });
    }
}
```

Quand on sélectionne l'image, cela l'affiche :

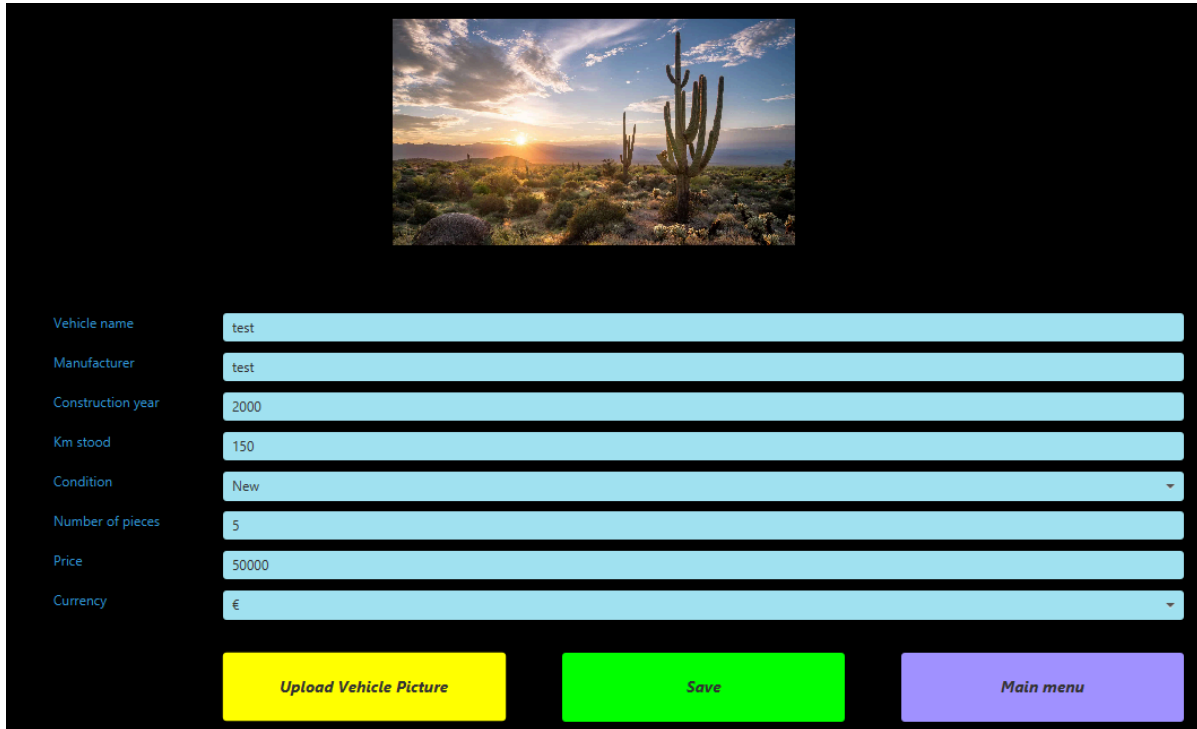


Exercice 9

On remplit les listes déroulantes d'option au défaut :

```
cb_condition.getItems().addAll( ...es: "New", "Used");  
cb_currency.getItems().addAll( ...es: "€", "$", "£");
```

Ajout de la sauvegarde avec le bouton **“Save”** :



The screenshot shows a web form for adding a vehicle. At the top, there is a large image of a desert landscape with saguaro cacti at sunset. Below the image, the form contains the following fields:

- Vehicle name: test
- Manufacturer: test
- Construction year: 2000
- Km stood: 150
- Condition: New (dropdown menu)
- Number of pieces: 5
- Price: 50000
- Currency: € (dropdown menu)

At the bottom of the form, there are three buttons: "Upload Vehicle Picture" (yellow), "Save" (green), and "Main menu" (purple).

vehicle_id	vehicle_name	manufacturer	construction_year	km_stood	vehicle_condition	pieces	price	currency	availability	picture
1	test	test	2000	150	New	5	50000	€	Available	[BLOB - 256.0 KiB]

(Parti du code du bouton “Save”) :

```
try {
    double input_kmStood = Double.parseDouble(input_km_stood_str);
    double input_price = Double.parseDouble(input_price_str);
    int input_pieces = Integer.parseInt(input_pieces_str);
    FileInputStream fileInputStream = new FileInputStream(path);
    java.sql.Connection con = DBConnection.getConnection();
    String sql = "INSERT INTO vehicle (vehicle_name, manufacturer, construction_year, km_stood, vehicle_condition, pieces, price, currency, availability, picture) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    java.sql.PreparedStatement pre = con.prepareStatement(sql);

    pre.setString(1, input_vehicle_name);
    pre.setString(2, input_manufacturer);
    pre.setString(3, input_construction_year);
    pre.setDouble(4, input_kmStood);
    pre.setString(5, selection_condition);
    pre.setInt(6, input_pieces);
    pre.setDouble(7, input_price);
    pre.setString(8, select_currency);
    pre.setString(9, "x" + "Available");
    pre.setBlob(10, fileInputStream);
    pre.executeUpdate();
    con.close();
    FXMLLoader loader = new FXMLLoader(HelloApplication.class.getResource("name: vehicletable.fxml"));
    javafx.scene.Parent root = loader.load();
    Stage stage = (Stage) button_save.getScene().getWindow();
    stage.setScene(new javafx.scene.Scene(root, w: 1200, h: 900));
    stage.setMaxWidth(1200);
    stage.setMaxHeight(900);
    stage.setTitle("Vehicle Table");
    stage.show();
} catch (java.sql.SQLException | java.io.IOException e) {
    throw new RuntimeException(e);
} catch (NumberFormatException e) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Format Error");
    alert.setContentText("Please enter valid numbers for Km, Price, and Pieces.");
    alert.show();
}
```

Exercice 10

```
button_main_menu.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        try {
            FXMLLoader loader = new FXMLLoader(HelloApplication.class.getResource("name: hello-view.fxml"));
            javafx.scene.Parent root = loader.load();
            Stage stage = (Stage) button_main_menu.getScene().getWindow();
            stage.setScene(new javafx.scene.Scene(root, w: 1200, h: 900));
            stage.setTitle("Welcome");
            stage.show();
        } catch (java.io.IOException e) {
            throw new RuntimeException(e);
        }
    }
});
```

Le bouton “Main menu” ramène belle est bien au menu principale grâce au code

Exercise 11

Création de la classe modèle “*ModelTableVehicle.java*” :

```
package org.banquetd7.bvl_s2_b2tp8;

public class ModelTableVehicle {
    private String vehicle_id, vehicle_name, manufacturer, construction_year, km_stood;
    public ModelTableVehicle(String vehicle_id, String vehicle_name, String manufacturer, String construction_year, String km_stood, String vehicle_condition, String pieces, String price, String currency, String availability) {
        this.vehicle_id = vehicle_id;
        this.vehicle_name = vehicle_name;
        this.manufacturer = manufacturer;
        this.construction_year = construction_year;
        this.km_stood = km_stood;
        this.vehicle_condition = vehicle_condition;
        this.pieces = pieces;
        this.price = price;
        this.currency = currency;
        this.availability = availability;
    }

    public String getVehicle_id() { return vehicle_id; } no usages
    public String getVehicle_name() { return vehicle_name; } no usages
    public String getManufacturer() { return manufacturer; } no usages
    public String getConstruction_year() { return construction_year; } no usages
    public String getKm_stood() { return km_stood; } no usages
    public String getVehicle_condition() { return vehicle_condition; } no usages
    public String getPieces() { return pieces; } no usages
    public String getPrice() { return price; } no usages
    public String getCurrency() { return currency; } no usages
    public String getAvailability() { return availability; } no usages
}
```


Création de “**TableView.java**” qui va gérer le tableau (extrait) :

```
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_id; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_vehicle_name; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_manufacturer; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_construction_year; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_km_stood; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_condition; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_pieces; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_price; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_currency; 2 usages
@FXML private TableColumn<ModelTableVehicle, String> tablecolumn_availability; 2 usages
@FXML private Button button_main_menu; 2 usages

PreparedStatement pre = null; 2 usages
ResultSet rs = null; 12 usages
ObservableList<ModelTableVehicle> observableList = FXCollections.observableArrayList(); 3 usages

@Override
public void initialize(URL url, ResourceBundle resourceBundle) {

    tablecolumn_id.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_vehicle_name.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_manufacturer.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_construction_year.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_condition.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_km_stood.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_price.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_pieces.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_currency.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");
    tablecolumn_availability.setStyle("-fx-background-color:black; -fx-text-fill:#00aaff; -fx-font-size:16");

    try {
        Connection con = DBConnection.getConnection();
        pre = con.prepareStatement("sql: \"SELECT * FROM vehicle\"");
        rs = pre.executeQuery();
        while (rs.next()) {
            observableList.add(new ModelTableVehicle(
                rs.getString(columnLabel: "vehicle_id"), rs.getString(columnLabel: "vehicle_name"),
                rs.getString(columnLabel: "manufacturer"), rs.getString(columnLabel: "construction_year"),
                rs.getString(columnLabel: "km_stood"), rs.getString(columnLabel: "vehicle_condition"),
                rs.getString(columnLabel: "pieces"), rs.getString(columnLabel: "price"),
                rs.getString(columnLabel: "currency"), rs.getString(columnLabel: "availability")
            ));
        }
        System.out.println(observableList);
    }
```

ID	Vehicle name	Manufacturer	Construction year	Km stood	Condition	Pieces	Price	Currency	Availability
2	trgdf	rgdfbc	2003	200.0	Used	50	20.0	\$	Available
1	test	test	2000	150.0	New	5	50000.0	€	Available

Main menu

Je dois modifier car l’affichage est mauvais

je dois ajouter

`“tableview_vehicle.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY);”`

pour que la taille soit automatiquement adapter

ID	Vehicle name	Manufacturer	Construction year	Km stood	Condition	Pieces	Price	Currency	Availability
1	test	test	2000	150.0	New	5	50000.0	€	Available
2	trgdf	rgdfbc	2003	200.0	Used	50	20.0	\$	Available
3	y	y	2000	20.0	New	5	10.0	€	Available

Main menu

Exercice 12

```
public static Integer id_number; no usages
public static String vehicule_name, manufacturer, construction_year, condition, currency, availability;
public static Double kmstood, price; no usages
public static Integer pieces; no usages
public static java.sql.Blob blob; no usages
```

Préparation du ClickEvent quand on cliquera sur une des lignes du tableau :

```
// q12
tableview_vehicle.setOnMouseClicked(new EventHandler<MouseEvent>() {
    @Override
    public void handle(MouseEvent mouseEvent) {
        // pas de crash si on clique dans le vide
        if (tableview_vehicle.getSelectionModel().getSelectedItem() != null) {

            id_number = Integer.valueOf(tableview_vehicle.getSelectionModel().getSelectedItem().getVehicle_id());
            System.out.println(id_number + " id_number");

            try {
                Connection con = DBConnection.getConnection();
                // récup des donné avec son id
                pre = con.prepareStatement( sql: "SELECT * FROM vehicle WHERE vehicle_id = ?");
                pre.setInt( parameterIndex: 1, id_number);
                rs = pre.executeQuery();

                while(rs.next()){
                    vehicule_name = rs.getString( columnLabel: "vehicule_name");
                    manufacturer = rs.getString( columnLabel: "manufacturer");
                    construction_year = rs.getString( columnLabel: "construction_year");
                    kmstood = rs.getDouble( columnLabel: "km_stood");
                    condition = rs.getString( columnLabel: "vehicle_condition");
                    pieces = rs.getInt( columnLabel: "pieces");
                    price = rs.getDouble( columnLabel: "price");
                    currency = rs.getString( columnLabel: "currency");
                    availability = rs.getString( columnLabel: "availability");
                    blob = rs.getBlob( columnLabel: "picture");
                }

                FXMLLoader loader = new FXMLLoader(VehicleTable.class.getResource( name: "selectedvehicle.fxml"));
                Parent root = loader.load();
                Stage stage = (Stage) ((Node) mouseEvent.getSource()).getScene().getWindow();

                stage.setScene(new Scene(root, v: 1200, vt: 900));
                stage.setMaxHeight(900);
                stage.setMaxWidth(1200);
                stage.setTitle("Selected Vehicle");
                stage.show();

            } catch (SQLException | IOException e) {
                throw new RuntimeException(e);
            }
        }
    }
});
```

(La ligne 5 est sélectionnée)

ID	Vehicle name	Manufacturer	Construction year
1	test	test	2000
2	trgdf	rgdfbc	2003
3	y	y	2000
4	r	r	2002
5	grdfcbv	ghfcv	1500

(C'est pour l'instant vide)



Exercice 13

Création de la page où on aura les détails d'un véhicule :

```
<AnchorPane prefHeight="900.0" prefWidth="1200.0" style="-fx-background-color: black;" xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1" fx:controller="selectedvehicle.fxml">

<children>



<Label alignment="CENTER" layoutY="20.0" prefWidth="1200.0" text="Update Vehicle Data" textFill="#00aaff">



<font>



<Font name="System Bold Italic" size="45.0" />



</font>



</Label>



<ImageView fx:id="imageview_vehicle_picture" fitHeight="250.0" fitWidth="500.0" layoutX="350.0" layoutY="100.0" pickOnBounds="true" preserveRatio="true" />



<VBox layoutX="100.0" layoutY="380.0" prefWidth="200.0" spacing="15.0">



<children>



<Label prefHeight="30.0" text="Vehicle ID" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Vehicle name" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Manufacturer" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Construction year" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Km stood" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Condition" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Number of pieces" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Price" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="Currency" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



<Label prefHeight="30.0" text="AvailAbility" textFill="#00aaff"><font><Font name="System Bold" size="16.0" /></font></Label>



</children>



</VBox>



<VBox layoutX="300.0" layoutY="380.0" prefWidth="800.0" spacing="15.0">



<children>



<TextField fx:id="tf_vehicle_id" prefHeight="30.0" editable="false" style="-fx-background-color: #b3ffff;" />



<TextField fx:id="tf_vehicle_name" prefHeight="30.0" style="-fx-background-color: #b3ffff;" />



<TextField fx:id="tf_manufacturer" prefHeight="30.0" style="-fx-background-color: #b3ffff;" />



<TextField fx:id="tf_construction_year" prefHeight="30.0" style="-fx-background-color: #b3ffff;" />



<TextField fx:id="tf_km_stood" prefHeight="30.0" style="-fx-background-color: #b3ffff;" />



<TextField fx:id="tf_condition" prefHeight="30.0" style="-fx-background-color: #b3ffff;" />



</children>



</VBox>



</AnchorPane>


```

Voici le résultat visuelle :

Selected Vehicle

Update Vehicle Data

Vehicle ID	
Vehicle name	
Manufacturer	
Construction year	
Km stood	hhhhhhhhhh
Condition	hh
Number of pieces	25hh
Price	78524
Currency	hhhh
Availability	

Upload Picture

Save Picture

Update

Delete

Table Vehicle

Exercise 14

Création de “SelectedVehicle.java” :

 SelectedVehicle

```
@Override
public void initialize(URL url, ResourceBundle resourceBundle) {

    try {
        if (VehicleTable.blob != null) {
            InputStream inputStream = VehicleTable.blob.getBinaryStream();
            Image image = new Image(inputStream);
            imageview_vehicle_picture.setImage(image);
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

    tf_vehicle_id.setText(String.valueOf(VehicleTable.id_number));
    tf_vehicle_name.setText(VehicleTable.vehicule_name);
    tf_manufacturer.setText(VehicleTable.manufacturer);
    tf_construction_year.setText(VehicleTable.construction_year);
    tf_km_stood.setText(String.valueOf(VehicleTable.kmstood));
    tf_condition.setText(VehicleTable.condition);
    tf_pieces.setText(String.valueOf(VehicleTable.pieces));
    tf_price.setText(String.valueOf(VehicleTable.price));
    tf_currency.setText(VehicleTable.currency);
    tf_availability.setText(VehicleTable.availability);
}
```

Voici le résultat (ce sont des valeurs de test) :

Selected Vehicle

Update Vehicle Data



Vehicle ID	7
Vehicle name	201
Manufacturer	2401
Construction year	5420
Km stood	4102021.0
Condition	Used
Number of pieces	5201
Price	21.0
Currency	€
Availability	Available

Upload Picture

Save Picture

Update

Delete

Table Vehicle

Pas d'exercice 15

Exercice 16

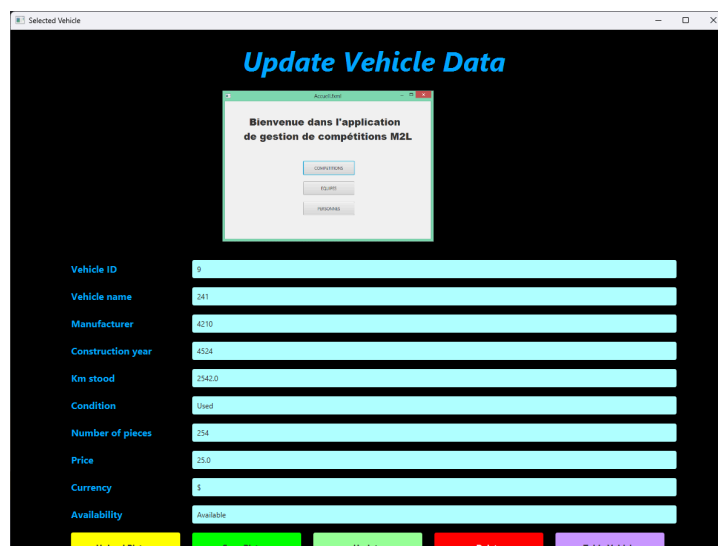
On doit faire fonctionner le bouton “**Upload Picture**”

La ligne “***String path;***” permet d'initialiser un lien de fichier/dossier.

C'est similaire à l'exercice 8 :

```
button_upload_picture.setOnAction(new EventHandler<javafx.event.ActionEvent>() {
    @Override
    public void handle(javafx.event.ActionEvent event) {
        javafx.stage.FileChooser fileChooser = new javafx.stage.FileChooser();
        fileChooser.setTitle("Upload Vehicle Picture");
        fileChooser.getExtensionFilters().addAll(
            new javafx.stage.FileChooser.ExtensionFilter("Image Files", ...strings: "*.jpg", "*.png", "*.jpeg", "*.svg")
        );
        Stage stage = (Stage) button_upload_picture.getScene().getWindow();
        java.io.File file = fileChooser.showOpenDialog(stage);
        if (file == null) {
            return;
        } else {
            try {
                path = file.getAbsolutePath();
                System.out.println(path);
                Image image = new Image(new java.io.FileInputStream(path));
                imageview_vehicle_picture.setImage(image);
            } catch (java.io.FileNotFoundException e) {
                throw new RuntimeException(e);
            }
        }
    }
});
```

(Image de test) (elle n'est pas sauvegardé dans la **BDD**)



Exercice 17

On doit faire fonctionner le bouton **“Save Picture”** comme ceci :

```
button_save_picture.setOnAction(new EventHandler<javafx.event.ActionEvent>() {
    @Override
    public void handle(javafx.event.ActionEvent event) {

        if (path == null) {
            javafx.scene.control.Alert alert = new javafx.scene.control.Alert(javafx.scene.control.Alert.AlertType.ERROR);
            alert.setContentText("Please select Vehicle picture file !");
            alert.setTitle("Warning");
            alert.show();
        } else {
            try {
                java.io.FileInputStream fileInputStream = new java.io.FileInputStream(path);
                java.sql.Connection con = DBConnection.getConnection();
                java.sql.PreparedStatement pre = con.prepareStatement("UPDATE vehicle SET picture = ? WHERE vehicle_id = ?");
                pre.setBlob(1, fileInputStream);
                pre.setInt(2, VehicleTable.id_number);
                pre.executeUpdate();

            } catch (java.sql.SQLException | java.io.FileNotFoundException e) {
                throw new RuntimeException(e);
            }
        }
    }
});
```

Maintenant quand le bouton **“Save Picture”** est pressé, cela mettra à jour la photo dans la BDD)

Exercice 18

On doit faire fonctionner le bouton “Delete” comme ceci :

```
// Q18
button_delete.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
        alert.setTitle("Warning");
        alert.setContentText("Are you sure ? Do you really want to delete this vehicle ?");
        Optional<ButtonType> result = alert.showAndWait();

        if (result.get() == ButtonType.CANCEL) {
            return;
        } else if (result.get() == ButtonType.OK) {
            try {
                Connection con = DBConnection.getConnection();

                PreparedStatement pre = con.prepareStatement("DELETE FROM vehicle WHERE vehicle_id = ?");
                pre.setInt(1, VehicleTable.id_number);
                pre.executeUpdate();
                FXMLLoader loader = new FXMLLoader(VehicleTable.class.getResource("vehicletable.fxml"));
                Parent root = loader.load();
                Stage stage = (Stage) button_delete.getScene().getWindow();
                stage.setScene(new Scene(root, 1200, 900));
                stage.setMaxWidth(1200);
                stage.setMaxHeight(900);
                stage.setTitle("vehicle table");
                stage.show();
            } catch (SQLException e) {
                throw new RuntimeException(e);
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        }
    }
});
```

On doit aussi faire fonctionner le bouton “Table Vehicule” qui est l'équivalent d'un bouton retour.

```
button_table_vehicle.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        try {
            FXMLLoader loader = new FXMLLoader(SelectedVehicle.class.getResource("vehicletable.fxml"));
            Parent root = loader.load();
            Stage stage = (Stage) button_table_vehicle.getScene().getWindow();
            stage.setScene(new Scene(root, 1200, 900));
            stage.setTitle("Vehicle Table");
            stage.show();
        } catch (IOException e) {
            throw new RuntimeException("Erreur de changement du fichier FXML : " + e.getMessage());
        }
    }
});
```

Exercice 19

On doit ensuite faire fonctionner le bouton “**Update**” qui permettra de mettre à jour les données dans la **BDD**.

```
button_update.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        try {
            Connection con = DBConnection.getConnection();
            PreparedStatement pre = con.prepareStatement("UPDATE vehicle SET vehicle_name = ?, manufacturer = ?, " +
                "construction_year = ?, km_stood = ?, vehicle_condition = ?, pieces = ?, " +
                "price = ?, currency = ?, availability = ? WHERE vehicle_id = ?");
            String name = tf_vehicle_name.getText();
            String manufacturer = tf_manufacturer.getText();
            String construction_year = tf_construction_year.getText();
            Double km_stood = Double.valueOf(tf_km_stood.getText());
            String condition = tf_condition.getText();
            Integer pieces = Integer.valueOf(tf_pieces.getText());
            Double price = Double.valueOf(tf_price.getText());
            String currency = tf_currency.getText();
            String availability = tf_availability.getText();
            String id_number = tf_vehicle_id.getText();
            pre.setString(1, name);
            pre.setString(2, manufacturer);
            pre.setString(3, construction_year);
            pre.setDouble(4, km_stood);
            pre.setString(5, condition);
            pre.setInt(6, pieces);
            pre.setDouble(7, price);
            pre.setString(8, currency);
            pre.setString(9, availability);
            pre.setString(10, id_number);
            pre.executeUpdate();
            FXMLLoader loader = new
                FXMLLoader(VehicleTable.class.getResource("vehicletable.fxml"));
            Parent root = loader.load();
            Stage stage = (Stage)
                button_update.getScene().getWindow();
            stage.setScene(new Scene(root, 1200, 900));
            stage.setMaxWidth(1200);
            stage.setMaxHeight(900);
            stage.setTitle("vehicle table");
            stage.show();
        } catch (SQLException e) {
            throw new RuntimeException(e);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
});
```

l'id n'est évidemment pas modifiable.

Exercice 20

Il fallait faire la même procédure pour les vendeurs et les clients, j'ai utilisé ce qui avait été fait auparavant avec les véhicules pour faire le reste.

(Les fichiers seront disponible dans le portfolio)

Bonus

Il nous été demandé de créer un .jar (sur IntelliJ) :

"File | Project Structure... | Artifacts" ->

cliquer sur le "+" -> sélectionner "JAR ->

From modules with dependencies..." ->

sélectionner la "Main Class" (point d'entrée de l'application) ->

cliquer sur "OK" ->

dans la fenêtre suivante, cliquer sur "Apply", puis "OK" ->

dans la barre de menus, faire "Build | Build Artifacts..." ->

dans la liste "Action", sélectionnez "Build" ->

le fichier "jar" est généré dans le répertoire

"out/artifacts/nom_du_projet_jar/".

Conclusion

Ce TP m'a permis d'apprendre concrètement à faire communiquer une interface JavaFX avec une base de données MySQL. J'ai pu mettre en pratique les requêtes SQL préparées, la gestion des événements utilisateurs et le stockage d'images via le type BLOB. Appliquer la même logique fonctionnelle aux véhicules, puis aux clients et vendeurs, m'a démontré l'importance d'écrire un code modulaire et réutilisable.